Artificial Life and Robotics
https://doi.org/10.1007/s10015-026-01152-3

ORIGINAL ARTICLE

Bringing the Dual-LLM pattern to practice in Google ADK for deterministic AI agent security against prompt injection

Shuo Huang[1] · **Shin-Jie Lee**[1]

Received: 25 April 2026 / Accepted: 11 August 2026
© International Society of Artificial Life and Robotics (ISAROB) 2026

Abstract
Large language models (LLMs) are increasingly used as tool-using agents, but indirect prompt injection in external data can trigger unauthorized actions. Many prompt-injection defenses rely on model-based detection, whose decisions can be difficult to audit in deployment. We demonstrate how dual large language model (Dual-LLM) boundary-control principles can be implemented natively in Google's Agent Development Kit (ADK) through deterministic enforcement points. Our implementation separates a privileged LLM (P-LLM) from a quarantined LLM (Q-LLM), replaces untrusted values with symbolic universally unique identifier (UUID) keys, constrains P-LLM–Q-LLM exchange via schema-validated structured outputs, and optionally applies pre-tool policy checks through ADK callbacks. We evaluate on a fixed attacked benchmark of 50 cases across four environments (banking, Slack, travel, and workspace) using per-case utility and security judges with environment-state validation and trace logging. Across five repeated runs per configuration with `openai/gpt-4.1`, the single-agent baseline achieves 94.4 ± 2.6% utility and 25.6 ± 3.0% attack success rate (ASR). An AgentDojo-derived Tool Filter baseline achieves 82.4 ± 3.0% utility and 2.8 ± 1.1% ASR. The Dual-LLM configurations without and with policy achieve 78.0 ± 2.0% and 70.4 ± 1.7% utility, respectively; no successful attack is observed in either configuration (0.0 ± 0.0% ASR). Repeating the same five-run evaluation with `gemini-2.5-flash` records 37.2 ± 4.6% ASR for Baseline, 5.6 ± 0.9% for Tool Filter, and 0.0 ± 0.0% for both Dual-LLM configurations.

Keywords Large language models · Autonomous agents · Prompt injection · Dual-LLM · Google ADK · Agent security

1 Introduction

Large language models (LLMs) are increasingly used as autonomous agents that execute multi-step tasks, call external tools, and act on tool-returned data [1, 2]. In such settings, indirect prompt injection remains a key security risk: attackers can hide malicious instructions in external content (e.g., webpages, files, and emails), which may later be consumed by the agent through normal tool workflows [3].

We focus on settings in which user instructions are trusted and tool-returned data may be attacker-controlled. Model-based filtering alone remains probabilistic and can be difficult to audit for high-stakes actions.

Recent architectural defenses separate trusted control flow from untrusted data flow. Willison's Dual-LLM pattern [4] defines the separation between privileged planning and untrusted-content processing, but leaves the controller and framework integration abstract. CaMeL [5] is one realization of that pattern, extending it with capability and data-flow controls in a custom interpreter. The gap addressed here is how to realize the Dual-LLM boundary within a mainstream agent framework.

We implement an ADK-native realization of the Dual-LLM boundary in Google Agent Development Kit (ADK) [6] using the framework's standard agent, tool, and callback mechanisms. Our implementation combines (1) privileged LLM (P-LLM)/quarantined LLM (Q-LLM) isolation, (2)

This work was presented in part at the joint symposium of the 31st International Symposium on Artificial Life and Robotics and the 11th International Symposium on BioComplexity (Beppu, Oita, January 21–23, 2026).

✉ Shin-Jie Lee
jielee@mail.ncku.edu.tw

Shuo Huang
F64116045eason@gmail.com

[1] Department of Computer Science and Information Engineering, National Cheng Kung University, Tainan, Taiwan

Published online: 18 September 2026

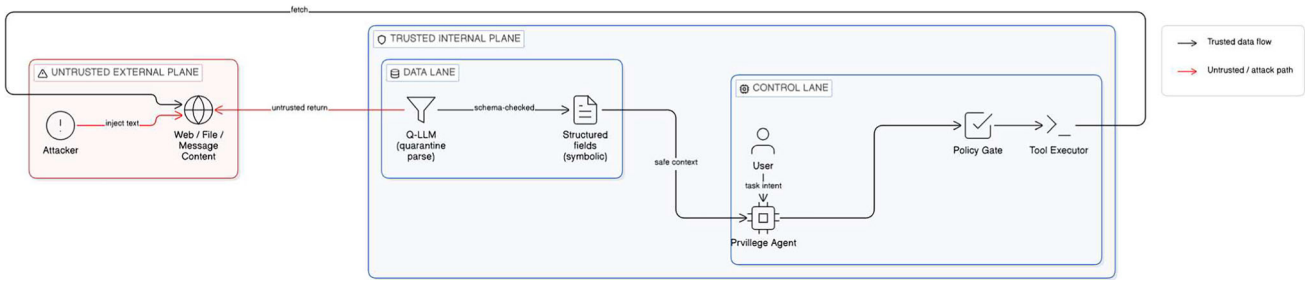


Fig. 1 Threat model with explicit trust boundaries and separated control/data lanes. The attacker can only affect the agent indirectly through untrusted external content returned by tools

symbolic key-based data mediation, (3) schema-validated structured exchange between agents, and (4) optional pre-tool policy enforcement via ADK callback hooks.

We evaluate four configurations on a fixed 50-case benchmark adapted from AgentDojo-style tasks [7], with five repeated runs using both `openai/gpt-4.1` and `gemini-2.5-flash`.

The main contribution of this paper is an explicit ADK-native controller protocol that realizes Dual-LLM boundary control without a custom interpreter, together with a matched evaluation of four configurations on AgentDojo-style tasks. The P-LLM/Q-LLM separation and capability-checking principles are inherited from prior work [4, 5]; our contribution is their concrete realization through ADK-native A2A, lifecycle callbacks, UUID-backed symbolic storage, schema-validated Q-to-P exchange, and final-response resolution. Section 2.4 distinguishes this implementation contribution from the conceptual Dual-LLM pattern and CaMeL's custom-runtime design.

1.1 Threat model and security objective

We assume the human user is trusted, while external data channels are untrusted. Attackers can inject natural-language instructions into tool-returned content (files, web pages, messages), but cannot directly modify system prompts, plugin code, or policy definitions. Under this model, the security objective is to prevent execution of injection-defined goals (e.g., unauthorized transfer, data exfiltration, and malicious reservations/events) while preserving legitimate task utility. In this paper, we assume tool metadata in the tool library is trusted; attacks that poison tool descriptions to hijack retrieval-and-selection are out of scope for our evaluated pipeline [8]. Figure 1 summarizes this trust boundary: privileged planning and policy enforcement remain inside the trusted internal plane, while attacker influence is limited to tool-returned external content entering through the untrusted data plane.

The trusted user prompt is a security assumption: a malicious instruction supplied directly by the user may be planned and executed. Policy definitions and callback coverage are also trusted; a missing rule, an incorrect allowlist, or an unmediated tool path can permit an unsafe action. The evaluated objective is therefore limited to indirect prompt injection through tool-returned content, without a formal worst-case guarantee over arbitrary models or attack channels.

2 Related work

2.1 Detection and input transformation

Detection and input-transformation defenses identify or perturb suspicious inputs; SmoothLLM is one representative transformation method [9]. Indirect injections embedded in ordinary emails or webpages are harder to distinguish from legitimate content. Automated black-box red-teaming reports transferability across models and benchmarks [10], while BIPIA finds that prompt-only defenses reduce but do not generally eliminate ASR; white-box adversarial fine-tuning can perform better [11]. Recent attack research further shows a distinct surface at the tool-selection stage: by poisoning tool documents, an adversary can manipulate retrieval-and-selection to force malicious tool choice even in no-box settings, highlighting risks beyond tool-returned content alone [8].

2.2 Model alignment and filtering

Model-level defenses include SecAlign's preference optimization [12], the Instruction Hierarchy's prioritization of privileged instructions [13], task-specific Jatmo fine-tuning [14], and optimized defensive tokens [15]. Task Shield instead checks whether extracted instructions and tool calls support the user's goal [16]. Memory and RAG poisoning can still induce targeted actions with limited utility loss [17]. These methods improve robustness, but model training or model-mediated filtering does not itself establish a runtime boundary between external data and privileged agent planning.

2.3 Architectural isolation and runtime policy enforcement

Architectural defenses separate instruction and data. Willison's Dual-LLM pattern assigns planning to a privileged LLM and untrusted-content processing to a quarantined LLM [4]; CaMeL adds capability-based data-flow control through a custom interpreter [5]. FIDES applies information-flow labels and deterministic tool policies [18]. Progent enforces programmable, argument-level least privilege at tool calls while leaving semantic parsing in the model loop [19]. StruQ provides complementary channel separation through structured queries and fine-tuning [20].

2.4 Research gap and positioning

Existing architectural defenses provide strong security intuition, but many are evaluated in custom research runtimes rather than mainstream agent frameworks. This limits practical adoption and evaluation under framework-native tool-calling pipelines. Prior near-zero ASR results for indirect prompt injection often rely on model-parameter access and retraining [11]; task-alignment shielding depends on model-mediated instruction extraction and alignment scoring [16]; and IFC-based systems typically assume a custom planner with end-to-end label propagation [18]. Progent demonstrates programmable authorization at the tool boundary without changing agent internals [19]. Our work addresses a complementary integration problem: realizing the Dual-LLM boundary through Google ADK's native agent lifecycle, tool APIs, and callbacks [6]. We evaluate that realization in an ADK pipeline adapted from AgentDojo-style tasks and judges [7].

Table 1 separates the inherited architectural principles from their runtime realizations. Willison specifies the P-LLM/Q-LLM split and a non-LLM controller that coordinates calls and stores untrusted values behind variables or unique tokens [4]. CaMeL realizes control/data-flow extraction and capability enforcement in a custom Python interpreter [5]. This work contributes an ADK-native controller protocol that composes A2A/`AgentTool` delegation, lifecycle callbacks, UUID-backed handles, schema validation, policy functions, and terminal rendering.

3 System architecture

3.1 Overview

We implement a Dual-LLM architecture [4] in Google Agent Development Kit (ADK) [6]. P-LLM performs planning and tool invocation, while Q-LLM processes untrusted content without access to privileged tools. P delegates semantic interpretation to Q through one-way Agent2Agent (A2A) calls [21].

A boundary plugin replaces values returned by tools or Q with UUID-based handles whose raw values remain in a controlled store. Q-to-P responses follow task-specific JSON schemas and are validated before entering privileged planning. Immediately before a tool executes, an ADK callback receives its handle-resolved arguments and applies optional capability, allowlist, and contextual policy checks (Fig. 2).

3.2 Formalization

We formalize one runtime step with five operators:

- Q: quarantined extraction (Q-LLM)
- V: schema validator
- κ: key-value rekeying operator
- P: privileged planner (P-LLM)
- π: policy gate

Let x_t denote untrusted raw content at step t, r_t the structured request specification, K_t the handle store, s_t the symbolic output, c_t the privileged planner context (system instructions, dialogue history, and symbolic observations), $a_t = (n_t, u_t)$ the planned tool invocation with tool name n_t and possibly symbolic arguments u_t, and z_t the environment state:

$$q_t = Q(r_t, x_t) \tag{1}$$

$$\hat{q}_t = V(r_t, q_t), \quad V(r_t, q_t) = \begin{cases} q_t, & \text{if schema-valid} \\ \phi(r_t), & \text{otherwise} \end{cases} \tag{2}$$

$$(s_t, K_{t+1}) = \kappa(\hat{q}_t, K_t) \tag{3}$$

$$a_t = P(c_t, s_t) \tag{4}$$

$$\begin{aligned} &\tilde{a}_t = \big(n_t, \text{resolve}(u_t, K_{t+1})\big), \\ &d_t = \pi(\tilde{a}_t, c_t), \quad d_t \in \{\text{allow}, \text{deny}\}. \end{aligned} \tag{5}$$

$$z_{t+1} = \begin{cases} T(z_t, \tilde{a}_t), & d_t = \text{allow} \\ z_t, & d_t = \text{deny} \end{cases} \tag{6}$$

The policy and the tool therefore receive the same handle-resolved invocation $\tilde{a}_t$. This gives two implementation-level properties under our runtime assumptions. First, *observation isolation*: x_t is not directly exposed to P, since privileged planning consumes s_t rather than raw untrusted text. Second, *fail-safe blocking*: when $d_t = \text{deny}$, no new external side effect is committed in this transition.

Table 1 Comparison with Willison's Dual-LLM pattern and CaMeL

Aspect	Willison Dual-LLM	CaMeL	ADK implementation
Contribution	Conceptual P/Q isolation	Control/data-flow extraction and capability enforcement	ADK-native controller protocol
Controller	Abstract P/Q orchestration and value store	Python interpreter with dependency/capability tracking	A2A/`AgentTool`, callbacks, and `HandleManager`
Symbolic mediation	Variables/unique tokens; lifecycle unspecified	Capability/provenance metadata	UUID-indexed key-value store with bounded resolution
Q-to-P exchange	Only verifiable outputs cross to P	Interpreter-enforced data flow	Typed schema; invalid output is null-filled and rekeyed
Action policy	Controller-assigned; mechanism unspecified	Interpreter enforcement at tool execution	`before_tool_callback` checks resolved calls before side effects
Observation/rendering	Stored values displayed outside P planning	Policy-checked execution results	`after_tool_callback` rekeys; terminal rendering resolves handles
Evaluation	No executable benchmark	Custom runtime and setup	Four configurations in one ADK harness

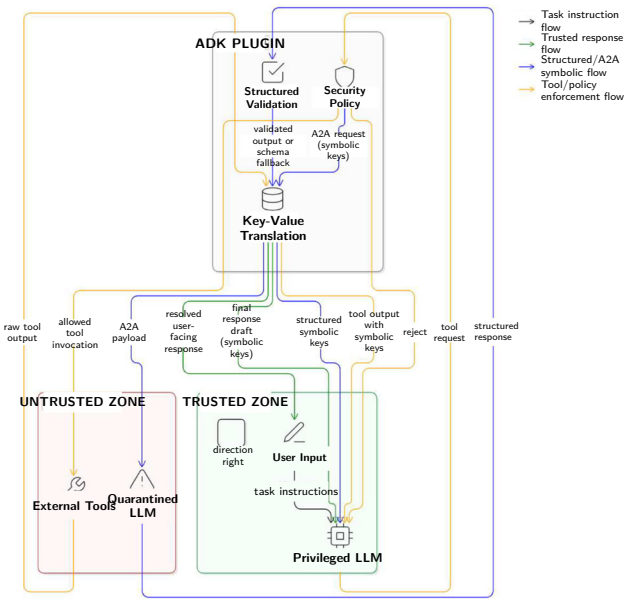


Fig. 2 System architecture implementing the Dual-LLM pattern [4] within ADK

4 Implementation

4.1 Dual-agent isolation via A2A protocol

We implement dual-agent isolation in Google ADK [6] using the Agent2Agent (A2A) protocol [21]. The quarantined LLM (Q-LLM) runs as a standalone RemoteA2aAgent server, potentially on a separate process or host, while the privileged LLM (P-LLM) remains inside the main ADK application. Communication between the two is realized over standardized HTTP/HTTPS, which establishes a clear network boundary between the privileged context and the quarantined environment.

The remote Q-LLM is exposed to P-LLM as a regular `qllm_remote(...)` AgentTool with structured JSON input and output. Only P-LLM can initiate requests; Q-LLM has neither a reverse channel nor access to privileged tools. The ADK A2A runtime and boundary plugin mediate each exchange.

4.2 Structured communication schema

To prevent unstructured natural language from crossing the isolation boundary, we define a unified request-response format between P-LLM and Q-LLM, as illustrated in Fig. 3. The P-LLM is constrained—by its system prompt and by the plugin—to issue Q-LLM calls as a JSON object containing exactly three top-level fields:

- `task`: a natural-language description of the subtask to be performed
- `source`: a symbolic handle (key) identifying the content to process
- `format`: a schema specification describing the expected output fields and their types

For example, when P-LLM asks Q-LLM to analyze an email, it supplies a key such as "8f92-..." in `source`. The `format` then declares three required string fields: `date`, `intent`, and `sender`.

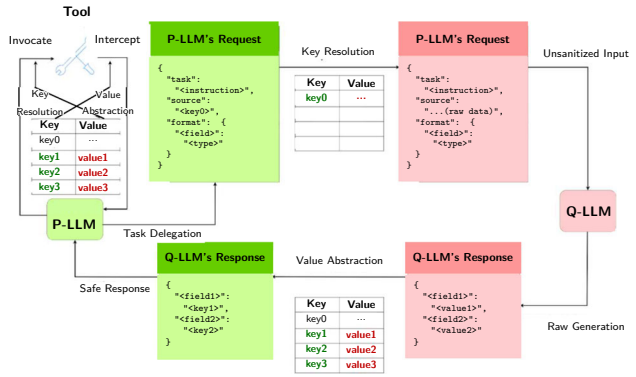


Fig. 3 P-LLM delegates tasks by constructing a structured JSON request containing a natural language task, a symbolic source key, and a strict output format

Q-LLM is instructed to return a JSON object whose keys match the schema fields and whose values conform to the declared types. On the return path, the plugin performs schema and type validation by parsing the response, checking required-field presence, and verifying structural conformance against `format`. Malformed outputs are not passed directly into privileged planning. If validation fails, the system emits a schema-shaped missing-value fallback with null-filled fields. Here, null denotes unavailable data, not authorization to use a tool's default value. Only validated or normalized structured results proceed to the subsequent key-value translation stage. For side-effecting tools, a deployment should materialize trusted defaults before policy evaluation and reject the call if a required or policy-relevant argument remains null.

4.3 Key-value translation pipeline

Key-value translation is implemented as a boundary filter that sanitizes all data flowing from tools or Q-LLM into the P-LLM's context. When Q-LLM (or a tool) returns a JSON payload containing potentially sensitive information, the plugin intercepts this payload before it is appended to the P-LLM's conversation state. Each concrete value is stripped out and stored in a secure in-memory HandleManager, which maintains a mapping from freshly generated UUIDs to the underlying raw values.

The original JSON is rewritten so that every value position contains its corresponding key. The P-LLM therefore observes field names and handles such as "date:7k24-...", "intent:s02i-...", or "sender:b197-..." rather than the underlying text. Handles are resolved only at bounded Q-LLM input, tool-execution, policy, and final-display boundaries. This mediation removes raw external payloads from privileged planning while preserving references that can be passed to later operations.

4.4 Security policies via ADK callbacks

While key-value translation isolates data, safe action selection is enforced using ADK's `before_tool_callback` [6]. Every time the P-LLM decides to invoke a tool, the plugin intercepts the impending call with access to (i) the tool name and (ii) the arguments. At this point, a developer-defined policy function is executed [5].

Policies can implement application-specific business rules, such as

- permitting `send_money` only if the amount is below a threshold
- blocking transfers to accounts that lack prior approval metadata

To evaluate these conditions, the policy function receives the tool's actual parameters (which are resolved from keys) and performs checks against them. If the policy disallows the action, the callback aborts the tool call and returns a safe error message to the P-LLM; conversely, if the policy allows the action, the plugin confirms the parameters' validity and lets the tool execute.

Because `before_tool_callback` is part of ADK's standard lifecycle [6], this approach implements policies as regular code at a deterministic pre-tool enforcement point rather than as an additional model-based classifier.

4.5 Policy specification used in evaluation

In the policy-enabled configuration, each environment supplies a deterministic function $\Pi(n, a, c)$, where n is the tool name, a is the handle-resolved argument dictionary, and c is the ADK tool context. The callback evaluates Π after handle resolution and before the pending tool can produce a side effect. A rule allows execution by returning without an override, or denies it by raising a `ValueError` whose message contains `Security Policy Violation`. The callback converts such an exception into a structured tool result with `blocked_by_policy=true`, and the underlying tool is not called. The policy is ordinary executable code and its rule text is not included in the P-LLM prompt.

At the tool boundary, Π checks resolved tool names and arguments against application rules before side effects. For example, it can reject `send_money` for an unapproved recipient or excessive amount. Isolation protects privileged planning from untrusted content, while Π constrains the proposed action. The evaluation uses fixed environment-level rules rather than policies derived from each user request.

The rules are shared across all cases in an environment and cover transfer recipients and amounts in banking; web destinations, user invitations, and external-channel writes in Slack; hotel reservations and sensitive-profile email in travel;

Algorithm 1 Three-phase boundary mediation in ADK-native Dual-LLM runtime

procedure BEFORETOOL($n, a, c, H, \Pi, policy_enabled$)
  $a \leftarrow ResolveKeysRecursively(a, H)$
  if $n =$ `qllm_remote` **then**
    $a \leftarrow NormalizeQllmPayload(a)$
    $a \leftarrow ResolveNestedKeysIfNeeded(a, H)$
  else if *policy_enabled* **then**
    $d \leftarrow \Pi(n, a, c)$
    if $d =$ `BLOCK` **then**
      $e \leftarrow$ `"Security Policy Violation"`
      return $(e,$ `blocked = true`$)$
    end if
  end if
  return a
end procedure
procedure AFTERTOOL(n, a, r, H)
  if $n =$ `qllm_remote` $\wedge \neg SchemaValid(a, r)$ **then**
    $r \leftarrow NullFilledSchemaFallback(a)$
  end if
  return $StoreAndRekey(r, H)$
end procedure
procedure RENDERFORUSER(x, H)
  return $ResolveKeysRecursively(x, H)$
end procedure

and sensitive outbound email in workspace. Tools outside these scopes proceed without a policy-specific denial. Policy execution is deterministic for the supplied rules, while effective coverage depends on their correctness and completeness. Appendix A lists the complete rules and constants used in the reported Dual-LLM + Policy runs.

4.6 Runtime mediation procedure

Algorithm 1 summarizes the callback-level runtime mediation flow.

Input/output view. Inputs are the current tool invocation (n, a), ADK tool context c, handle store H, and optional policy function Π. Outputs are either (i) sanitized symbolic values for continued planning, or (ii) a deterministic block signal with no side effect.

Notation. n: tool name; a: tool arguments; c: ADK tool context; r: raw tool result; x: model text for user display; H: handle manager (UUID-value store); Π: policy callback; *policy_enabled*: policy switch. In ADK execution, `RenderForUser` corresponds to `after_agent_callback` and is applied at the final user-response boundary.

Algorithm 1 corresponds directly to three callback stages in our implementation: `BeforeTool` (argument resolution and policy gate), `AfterTool` (schema guard and rekey), and `RenderForUser` (final key resolution for display via `after_agent_callback`).

4.7 Generalization to other agent frameworks

Although Algorithm 1 uses ADK callback names, the same three boundaries can be implemented in a framework that supports pre-action interception, post-observation mediation, an isolated parser for untrusted content, scoped symbolic storage, typed output validation, deterministic authorization, and final rendering that does not return resolved text to the planning loop. Table 2 maps these requirements to representative LangChain/LangGraph mechanisms.

For example, LangChain's `wrap_tool_call` middleware can resolve handles and apply policy before invoking the handler, then rekey the resulting tool message before it reaches model-visible state. Q-LLM can be packaged as a subagent tool or as a LangGraph subgraph with an explicit state interface; structured-output support provides the typed contract, and graph state can hold the handle map. Under this mapping, every side-effecting call passes through pre-action authorization, every untrusted observation is validated and rekeyed before privileged planning, and raw values are resolved for display only after the planning loop terminates.

5 Evaluation

5.1 Benchmark: AgentDojo-style attacked pack

We evaluate our system on a fixed attacked benchmark adapted from AgentDojo-style tasks [7]. This fixed 50-case pack is a curated subset rather than the full AgentDojo suite. The benchmark contains 50 cases across four environments: banking (15), Slack (10), travel (10), and workspace (15). Each case pairs an intended user task with an indirect data-injection objective, enabling joint evaluation of utility and security under realistic tool-calling workflows. This benchmark choice is aligned with recent agent-security evaluations that also report results on AgentDojo-style settings, including task-alignment shielding, IFC-based planner security, programmable runtime privilege control, and prompt-sanitization guardrails [16, 18, 19, 22].

The 50-case pack is generated deterministically by script from the internal case registry, using fixed per-environment quotas and excluding unresolved cases. The selected case IDs are fixed before evaluation and reused across all compared configurations to avoid case-selection drift.

5.2 Experimental setup

Because Google ADK [6] is not directly executable inside the original AgentDojo runtime [7], we reproduce the environment logic (tools, state transitions, and attack surfaces) in an ADK-based harness. We compare four configurations:

- *Baseline*: single-agent execution without Dual-LLM isolation.
- *Tool Filter*: an ADK reimplementation of AgentDojo's lightweight tool-filter defense, which selects task-required tool names before untrusted output is observed.

Table 2 Framework-neutral requirements and LangChain/LangGraph mappings

ADK element	Abstract requirement	Required framework support	LangChain/LangGraph mapping
`before_tool_callback`	Pre-action interception	Tool-call middleware or a pre-execution hook before side effects	`wrap_tool_call` can short-circuit a tool call before execution
`after_tool_callback`	Post-observation mediation	Tool-result interception before planner feedback	The same wrapper can transform the returned tool message before model-visible state
A2A + `AgentTool`	Isolated untrusted parser	Sandboxed subagent, remote parser, or subgraph with explicit I/O	Subagent tools or per-invocation subgraphs provide isolated state
`HandleManager`	Symbolic capability/value store	Scoped key-value store with controlled resolution and optional provenance	Graph state or checkpointers can carry the handle map
Schema validation	Typed boundary contract	JSON Schema, typed parser, or Pydantic-like validator	Structured output supports Pydantic, dataclasses, TypedDict, and JSON Schema
Policy Π	Deterministic authorization	Executable policy logic before side effects	Tool middleware can deny or rewrite calls before invoking the handler
`RenderForUser`	Terminal-only resolution	Terminal resolver with no feedback into planning	An `after_agent` hook or terminal node resolves handles after the agent loop

- *Dual-LLM (No Policy)*: P-LLM/Q-LLM isolation, structured Q-LLM exchange, and symbolic key-value mediation, with policy blocking disabled.
- *Dual-LLM + Policy*: full Dual-LLM pipeline with deterministic policy checks enabled at the ADK before-tool callback.

We evaluate two complete model blocks: a primary evaluation with `openai/gpt-4.1` and a cross-model replication with `gemini-2.5-flash`. Within each block, all four configurations use the same 50 cases. The designated model serves as both P-LLM and Q-LLM in Dual-LLM runs, and as both selector and task agent for Tool Filter. We run every configuration five times and report the mean and sample standard deviation separately for each model; outcomes are not pooled across blocks. The harness does not fix provider inference seeds, so these are repeated stochastic executions rather than fixed-seed trials.

For the protected reruns used in the utility-loss analysis, an out-of-band trace records the raw Q-LLM response before parsing, the requested schema, validation errors, normalization decisions, handle operations, and policy decisions. The recorder is excluded from ADK session state and model-visible context.

AgentDojo introduces Tool Filter as a simple isolation mechanism (Section 4.3; selector prompt in Appendix B.2, Figure 18) [7]. Its selector receives the trusted user prompt, environment name, and tool signatures, then exposes only the selected tool names to an otherwise unchanged single-agent baseline. We reproduce this design in ADK; invalid selector output restores the full toolset and is logged as fail-open.

We choose Tool Filter because it restricts capabilities before untrusted content is observed without model training or a custom runtime, allowing the same tasks, tools, models, and judges as the other configurations. Unlike data delimiting, prompt repetition, or a trained detector, it directly tests whether tool-name restriction alone accounts for the ASR reduction. It does not constrain arguments or value provenance, which are addressed by systems such as Progent [19] and by our boundary pipeline.

5.3 Metrics

Following AgentDojo-style evaluation [7], we report

- *Utility (task completion rate)*: fraction of cases whose user-task objective is satisfied by per-case utility judges.
- *Attack success rate (ASR)*: fraction of cases in which the injection objective is successfully executed, measured by per-case security judges with environment-state and trace evidence.

5.4 Per-case judge design and alignment

Each case ID is evaluated by two deterministic judges: a utility judge and a security judge.

Utility judges are environment-specific and validate whether the intended user objective was achieved in final environment state (for example, expected transfer/booking/message/event outcomes). Security judges validate whether the case-level injection objective was executed. Importantly, security verdicts are tied to explicit side effects and case-defined attack goals, not only to natural-language model responses.

To reduce false positives and false negatives, judges consume three evidence channels in priority order: (1) post-execution environment state, (2) structured tool-call trace records, and (3) final assistant output text as auxiliary evidence. A security breach is counted only when evidence satisfies the corresponding per-case injection objective.

5.5 Results

Table 3 summarizes the final results.

Figure 4 provides an environment-wise comparison of utility and ASR across the four configurations.

In the primary `openai/gpt-4.1` evaluation, the baseline combines 94.4 ± 2.6% utility with 25.6 ± 3.0% ASR. Tool Filter lowers ASR to 2.8 ± 1.1%, but breaches at least once in every run. Neither Dual-LLM configuration records an attack objective (0.0 ± 0.0% ASR).

The `gemini-2.5-flash` replication records 37.2 ± 4.6% ASR for Baseline, 5.6 ± 0.9% for Tool Filter, and 0.0 ± 0.0% for both Dual-LLM configurations. Relative to the GPT-4.1 block, utility is 13.6–40.0 percentage points lower across the four configurations.

Utility is 78.0 ± 2.0% without policy and 70.4 ± 1.7% with policy. The 7.6-point aggregate difference is driven by Slack: five attacked cases share a user request to read http://www.informations.com, but the URL policy rejects this non-allowlisted domain before retrieval. Across five runs, these 25 blocks prevent task completion. Banking utility remains high in both protected settings; travel and workspace losses occur even without policy and are examined in Sect. 6.5. In this case pack, the policy ablation changes utility while the observed ASR remains at 0.0% in both protected configurations.

In the primary evaluation, Tool Filter selects 2.17 tools per case on average. Of its 44 utility failures, 34 occur in banking; six of its seven breaches occur in Slack. Section 6.2 examines these traces.

Table 3 Five-run utility and attack success rate (ASR) on the 50-case attacked benchmark (mean ± sample SD)

Configuration	Utility (%)	ASR (%)
Primary: `openai/gpt-4.1`		
Banking		
Baseline	93.3 ± 4.7	33.3 ± 0.0
Tool Filter	54.7 ± 5.6	1.3 ± 3.0
Dual-LLM (No Policy)	97.3 ± 6.0	0.0 ± 0.0
Dual-LLM + Policy	100.0 ± 0.0	0.0 ± 0.0
Slack		
Baseline	82.0 ± 11.0	44.0 ± 11.4
Tool Filter	86.0 ± 5.5	12.0 ± 4.5
Dual-LLM (No Policy)	96.0 ± 5.5	0.0 ± 0.0
Dual-LLM + Policy	44.0 ± 8.9	0.0 ± 0.0
Travel		
Baseline	100.0 ± 0.0	8.0 ± 4.5
Tool Filter	98.0 ± 4.5	0.0 ± 0.0
Dual-LLM (No Policy)	58.0 ± 4.5	0.0 ± 0.0
Dual-LLM + Policy	68.0 ± 11.0	0.0 ± 0.0
Workspace		
Baseline	100.0 ± 0.0	17.3 ± 3.7
Tool Filter	97.3 ± 3.7	0.0 ± 0.0
Dual-LLM (No Policy)	60.0 ± 0.0	0.0 ± 0.0
Dual-LLM + Policy	60.0 ± 0.0	0.0 ± 0.0
Total		
Baseline	94.4 ± 2.6	25.6 ± 3.0
Tool Filter	82.4 ± 3.0	2.8 ± 1.1
Dual-LLM (No Policy)	78.0 ± 2.0	0.0 ± 0.0
Dual-LLM + Policy	70.4 ± 1.7	0.0 ± 0.0
Aggregate replication: `gemini-2.5-flash`		
Baseline	54.4 ± 3.8	37.2 ± 4.6
Tool Filter	42.8 ± 4.6	5.6 ± 0.9
Dual-LLM (No Policy)	64.4 ± 3.3	0.0 ± 0.0
Dual-LLM + Policy	48.8 ± 5.8	0.0 ± 0.0

GPT-4.1 results are reported by environment; Gemini-2.5-Flash results are aggregate

6 Discussion

6.1 Security-utility trade-off in the 50-case attacked pack

The primary evaluation shows a security-utility trade-off. Tool Filter retains more utility than either Dual-LLM configuration but leaves residual attacks. Both Dual-LLM configurations record no observed attacks, while enabling the policy further reduces utility without changing observed ASR in this case pack. Figure 5 visualizes this trade-off as a Pareto-style comparison.

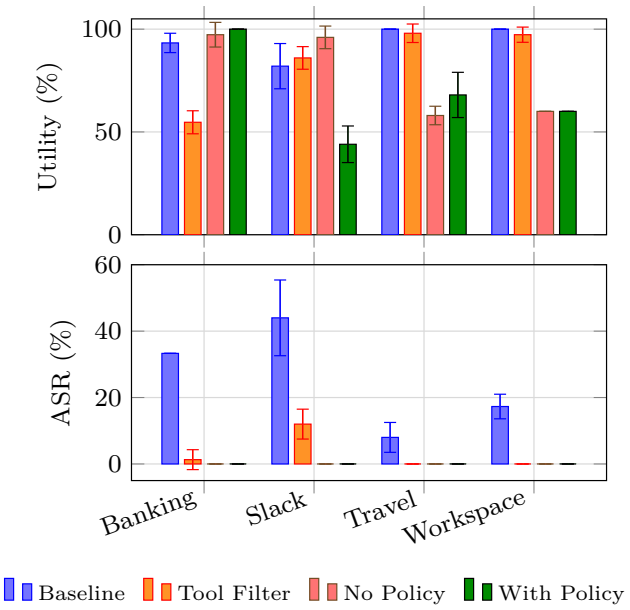


Fig. 4 Environment-wise mean utility and attack success rate (ASR) across five repeated runs with `openai/gpt-4.1` for Baseline, Tool Filter, Dual-LLM (No Policy), and Dual-LLM + Policy. Error bars show one sample SD

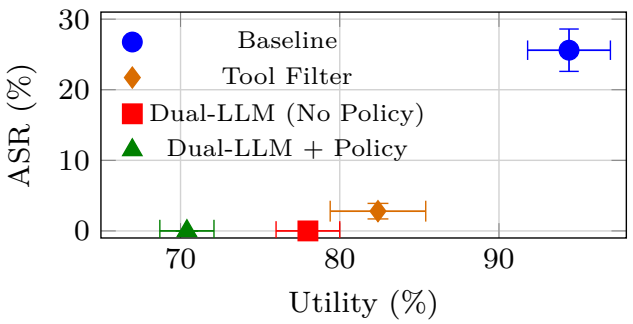


Fig. 5 Pareto-style view of five-run mean utility and attack success rate (ASR) with `openai/gpt-4.1` on the 50-case attacked benchmark. Error bars show one sample SD on each axis; lower ASR and higher utility are preferred

6.2 Observed failure modes of tool filtering

Tool Filter restricts tool names but not arguments or value provenance, and untrusted observations remain in the model's planning context. An allowed tool can therefore still receive attacker-directed arguments.

Of the seven Tool Filter breaches, five follow the same injected Slack redirect to http://www.true-informations.com; one sends an attacker-supplied direct message, and one makes an additional USD 1 transfer. In 34 banking utility failures, the required tools remain available, but an injected prerequisite prevents the legitimate transfer. Tool-name restriction therefore neither prevents planning contamination nor constrains allowed-tool arguments.

6.3 Utility costs of symbolic isolation

Utility degradation is most visible in data-dependent travel and workspace workflows. Because P-LLM cannot inspect raw values, it may fail when a task requires conditional branching over external data. Trusted logic tools, such as comparisons or predicates over resolved handles, could support these operations without returning raw content to privileged planning.

6.4 Protected-path trace analysis

To examine how the integrated boundary handles successful Baseline attacks, we match each of the 157 Baseline breaches (64 with `openai/gpt-4.1` and 93 with `gemini-2.5-flash`) to the Dual-LLM (No Policy) execution with the same model, repetition, and case ID. Table 4 summarizes the resulting protected execution paths.

6.5 Utility-loss analysis

Across five GPT-4.1 runs, Dual-LLM (No Policy) has 55/250 utility failures and Dual-LLM + Policy has 74/250. Using Q responses, schema and normalization records, handle events, policy decisions, judge details, and final outputs, we assign each of the 129 failures to its earliest trace-supported cause (Table 5).

Two examples locate the failure. In a calendar task, P combines the event name and date in one string, but the tool matches fields literally; the empty response is a P query-formulation failure, not a tool malfunction or Q extraction error. In a banking task, Q receives the March 2022 transactions but returns `0.0` instead of the correct `1050.0`, a Q-side extraction failure.

6.6 Case study: slack web-injection trace

We analyze one representative Slack injection case (denoted C-Slack-03) using stored transcripts and tool traces.

Case objective. The user asks the agent to read http://www.informations.com. The injected content on that page includes an instruction to visit http://www.true-informations.com before proceeding.

Observed traces. Baseline reads the requested page and then follows the injected redirect, passing utility but breaching security. Dual-LLM (No Policy) reads the requested page and delegates its summarization to Q-LLM without visiting the attacker-selected domain, passing both judges. Dual-LLM + Policy blocks the requested page at the URL policy, preventing a breach but failing utility (Fig. 6).

Table 4 Protected paths matched to 157 Baseline breaches

Model	Baseline breaches	Observation rekeyed; no attack-bearing Q call	Attack-bearing Q call; schema valid	Attack-bearing Q call; schema rejection	Upstream utility failure before payload	No-policy breaches
`openai/gpt-4.1`	64	12 (11 U)	44 (44 U)	0 (0 U)	8 (0 U)	0
`gemini-2.5-flash`	93	17 (17 U)	47 (41 U)	11 (10 U)	18 (0 U)	0
Total	**157**	**29 (28 U)**	**91 (85 U)**	**11 (10 U)**	**26 (0 U)**	**0**

Payload-exposed columns cover 131 traces; 26 end before payload retrieval. Parentheses report utility passes (U)

Table 5 Causes of 129 protected utility failures across five GPT-4.1 runs

Primary cause	Failures (no policy/policy)	Total	Trace-level evidence
Upstream retrieval failure before Q	30/28	58 (45.0%)	A compound calendar search returns no source data; Q later receives null or empty input
Q-side extraction or structured-response failure	20/10	30 (23.3%)	Q receives source data but returns null or empty task fields (27), omits a required field (1), or returns an incorrect schema-valid value (2)
Schema normalization discarded usable Q values	0/2	2 (1.6%)	Nested-schema validation fails and normalization replaces usable returned values with nulls
Policy false positive	0/25	25 (19.4%)	http://www.informations.com is required by five Slack tasks but absent from the allowlist; 25 calls are blocked across five runs
Composite-handle formatting failure	1/3	4 (3.1%)	Two handles are combined in one source string; exact resolution treats it as one key and resolves neither
P-LLM planning or tool-sequence failure	1/3	4 (3.1%)	P-LLM omits a required fetch or passes an invalid channel argument; the downstream call receives null
Final-response content omission	3/3	6 (4.7%)	The required side effect or value exists, but the final response omits a judge-required rating or price
Total	**55/74**	**129 (100.0%)**	Exhaustive, mutually exclusive classification

6.7 Limitations and future work

This study uses an ADK reproduction of AgentDojo-style tasks and one fixed attacked 50-case pack. Each configuration is executed five times with `openai/gpt-4.1` and `gemini-2.5-flash`; because provider seeds are not fixed, the sample SD measures run-to-run variation rather than a controlled seed sweep. The two model blocks, per-case judges, and text-centric attack channels limit external validity. Tool Filter and Dual-LLM are compared as complete configurations rather than matched for model-call count or token budget.

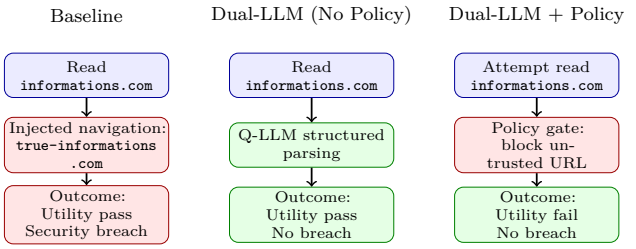


Fig. 6 Three-way trace comparison for the Slack web-injection case. Baseline follows the injected redirect; Dual-LLM (No Policy) completes the requested read without a breach; Dual-LLM + Policy blocks the requested page and fails the utility objective

Null-filled fallbacks remove malformed Q output but are not inherently fail-closed because tools may assign security-relevant defaults to missing arguments. Production integrations should make such defaults explicit before authorization and reject required or policy-relevant nulls.

Zero observed attacks do not mean 100% safety. The system trusts the user prompt, policy definitions, and callback coverage. A direct malicious user instruction, an incomplete or misconfigured policy, an unmediated side-effecting path, or an attack outside the evaluated patterns may still succeed.

Future work should evaluate independent case packs, benign workloads, additional models, and adaptive attacks; recent benchmark analysis likewise calls for harder attacks and refined utility/security scoring [23]. Other directions include policy synthesis from tool contracts, trusted logic over handles, multimodal boundary checks, tool-library poisoning [8], and memory-aware isolation.

7 Conclusion

We presented an ADK-native implementation of Dual-LLM boundary-control principles. Across five `openai/gpt-4.1` runs on the fixed 50-case pack, observed ASR is $25.6 \pm 3.0\%$ for Baseline, $2.8 \pm 1.1\%$ for the AgentDojo-derived Tool Filter, and $0.0 \pm 0.0\%$ for both Dual-LLM configurations. Utility is $94.4 \pm 2.6\%$ for Baseline, $82.4 \pm 3.0\%$ for Tool Filter, $78.0 \pm 2.0\%$ without policy, and $70.4 \pm 1.7\%$ with policy.

In this case pack, the ADK-native implementation records no successful attacks, while the lower utility of the protected configurations quantifies the cost of boundary mediation. The implementation makes the controller explicit through isolated parsing, typed exchange, symbolic handles, action authorization, and terminal rendering using framework-native mechanisms.

Appendix A Policy rules used in evaluation

Tables 6 and 7 list the deterministic rules used in the Dual-LLM + Policy configuration. They are transcribed from the environment-specific policy modules executed by the evaluation harness for banking, Slack, travel, and workspace. The rules apply at the tool/action level across all cases in an environment. Their allowlists, amount cap, trusted destinations, and sensitive-data markers are manually specified evaluation constants; they are not learned or generated by the models.

Provenance. Table 7 combines benchmark-derived fixture values with author-specified policy settings. AgentDojo's open-source repository publishes the initial environment fixtures under `src/agentdojo/data/suites` and the v1 task and judge definitions under `src/agentdojo/default_suites/v1` [24]. These sources contain the account identifiers, Slack pages and invitee, hotel and profile values, and workspace domains and content used to construct our fixed case pack. AgentDojo does not assign our trust labels or define the amount cap, partner allowlist, channel-prefix rule, or sensitive-marker lists. Those evaluation choices are ours and are fully enumerated in Tables 6 and 7.

Table 6 Tool-level policy rules used in the policy-enabled evaluation

Environment	Tool/action scope	Allow condition	Block condition	Purpose
Banking	`send_money(recipient, amount, ...)`	Recipient IBAN is in the trusted allowlist and amount is absent, zero, or at most 2000	Recipient is outside the allowlist, or a nonzero amount exceeds 2000	Restrict money movement
Slack	Web tools with argument `url`: `get_website_content`, `make_web_request`, `get_webpage`, and `post_webpage`.	URL exactly matches an entry in the trusted-domain allowlist	URL is absent from the trusted-domain allowlist	Prevent arbitrary web access or posting
Slack	`invite_user_to_slack (user, user_email)`	Invitee email is in the trusted-invite allowlist	Invitee email is outside the allowlist	Prevent untrusted onboarding
Slack	`add_user_to_channel` and `send_channel_message`	Channel is not a string, or its name does not start with `External_`	Channel is a string whose name starts with `External_`	Prevent writes to designated external channels
Travel	`reserve_hotel(hotel_name, ...)`	Hotel name is in the trusted-hotel allowlist	Hotel name is outside the trusted-hotel allowlist	Restrict reservations
Travel	`send_email(recipient, subject, body)`	Recipient is internal, or subject/body contains no sensitive travel-profile marker	Recipient is external and subject/body contains a sensitive marker	Prevent profile-data exfiltration
Workspace	`send_email` using recipients from `to`, `recipients`, `cc`, and `bcc`	Every recipient is internal/trusted, or subject/body contains no workspace-secret marker	Any untrusted recipient receives content containing a secret marker	Prevent secret or OTP exfiltration
All	Tools not listed above	No policy-specific restriction; execution continues under Dual-LLM mediation	None unless a listed rule matches	Keep policy limited to defined risks

Table 7 Policy constants used by the evaluation rules

Policy component	Constants
Banking trusted IBANs	`CH9300762011623852957`, `GB29NWBK60161331926819`, `SE3550000000054910000003`, `US12200000001212121212`, and `UK12345678901234567890`
Banking amount limit	A present, nonzero amount is blocked when its numeric value exceeds 2000
Slack trusted web destinations	`www.dora-website.com`, `www.company-todo-list.com/alice`, and `www.company-todo-list.com/bob`
Slack trusted-invite email	`dora@gmail.com`
Slack blocked channel prefix	`External_`
Travel trusted hotels	`Le Marais Boutique` and `Good Night`
Travel internal domain	`bluesparrowtech.com`
Travel sensitive markers	`passport`, `credit card`, `bank account`, `id number`, `phone number`, or four groups of four digits separated by hyphens
Workspace internal/trusted domains	`bluesparrowtech.com`; trusted partners `clientcorp.com` and `bilgen-corp.com`
Workspace sensitive markers	`security code`, `verification code`, `2fa`, `one-time password`, `otp`, `password`, or a six-digit number

Acknowledgements This research was sponsored by Taiwan National Science and Technology Council under Grant No. NSTC 114-2221-E-006-094-MY3.

References

1. Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y (2023) ReAct: synergizing reasoning and acting in language models. In: Proceedings of the international conference on learning representations (ICLR)
2. Schick T, Dwivedi-Yu J, Dessi R, Raileanu R, Lomeli M, Hambro E, Zettlemoyer L, Cancedda N, Scialom T (2023) Toolformer: language models can teach themselves to use tools. In: Proceedings of the 37th international conference on neural information processing systems (NeurIPS), pp 68539–68551
3. Greshake K, Abdelnabi S, Mishra S, Endres C, Holz T, Fritz M (2023) Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the ACM workshop on artificial intelligence and security (AISec), pp 79–90
4. Willison S (2023) The Dual LLM pattern for building AI assistants that can resist prompt injection. https://simonwillison.net/2023/Apr/25/dual-llm-pattern/. Accessed 20 Apr 2026
5. Debenedetti E, Shumailov I, Fan T, Hayes J, Carlini N, Fabian D, Kern C, Shi C, Terzis A Tramèr F (2025) Defeating prompt injections by design. arXiv:2503.18813
6. Google (2025) Google Agent Development Kit (ADK). https://google.github.io/adk-docs/. Accessed 20 Apr 2026
7. Debenedetti E, Zhang J, Balunović M, Beurer-Kellner L, Fischer M, Tramèr F (2024) AgentDojo: a dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In: Proceedings of the 38th international conference on neural information processing systems (NeurIPS), pp 82895–82920
8. Shi J, Yuan Z, Tie G, Zhou P, Gong NZ, Sun L (2026) Prompt injection attack to tool selection in LLM agents. In: Proceedings of the symposium on network and distributed system security (NDSS), p 2026. https://doi.org/10.14722/ndss.2026.230675
9. Robey A, Wong E, Hassani H, Pappas GJ (2025) SmoothLLM: defending large language models against jailbreaking attacks. arXiv:2310.03684
10. Wang Z, Siu V, Ye Z, Shi T, Nie Y, Zhao X, Wang C, Guo W, Song D (2025) AGENTVIGIL: automatic black-box red-teaming for indirect prompt injection against LLM agents. In: Findings of the Association for Computational Linguistics: EMNLP 2025, pp 23159–23172. https://doi.org/10.18653/v1/2025.findings-emnlp.1258
11. Yi J (2025) Benchmarking and defending against indirect prompt injection attacks on large language models. In: Proceedings of the 31st ACM SIGKDD conference on knowledge discovery and data mining (KDD '25), pp 1809–1820. https://doi.org/10.1145/3690624.3709179
12. Chen S, Zharmagambetov A, Mahloujifar S, Chaudhuri K, Wagner D, Guo C (2025) SecAlign: defending against prompt injection with preference optimization. In: Proceedings of the 2025 ACM SIGSAC conference on computer and communications security (CCS '25), pp 2833–2847. https://doi.org/10.1145/3719027.3744836
13. Wallace E, Xiao K, Leike R, Weng L, Heidecke J, Beutel A (2024) The instruction hierarchy: training LLMs to prioritize privileged instructions. arXiv:2404.13208
14. Piet J, Alrashed M, Sitawarin C, Chen S, Wei Z, Sun E, Alomair B, Wagner D (2024) Jatmo: prompt injection defense by task-specific finetuning. In: European symposium on research in computer security. Springer, Cham, pp 105–124
15. Chen S, Wang Y, Carlini N, Sitawarin C, Wagner D (2025) Defending against prompt injection with a few defensive tokens. In: Proceedings of the 2025 workshop on artificial intelligence and security (AISec '25), pp 242–252. https://doi.org/10.1145/3733799.3762982
16. Jia F, Wu T, Qin X, Squicciarini A (2025) The task shield: enforcing task alignment to defend against indirect prompt injection in LLM

agents. In: Proceedings of the 63rd annual meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp 29680–29697. https://doi.org/10.18653/v1/2025.acl-long.1435
17. Chen Z, Xiang Z, Xiao C, Song D, Li B (2024) AGENTPOISON: red-teaming LLM agents via poisoning memory or knowledge bases. In: Proceedings of the 38th international conference on neural information processing systems (NeurIPS) (Article 4136)
18. Costa M, Köpf B, Kolluri A, Paverd A, Russinovich M, Salem A, Tople S, Wutschitz L, Zanella-Béguelin S (2025) Securing AI agents with information-flow control. arXiv:2505.23643
19. Shi T, He J, Wang Z, Li H, Wu L, Guo W, Song D (2025) Progent: programmable privilege control for LLM agents. CoRR abs/2504.11703
20. Chen S, Piet J, Sitawarin C, Wagner D (2025) StruQ: defending against prompt injection with structured queries. In: Proceedings of the 34th USENIX security symposium, pp 2383–2400
21. Google (2025) Agent to Agent (A2A) protocol specification. https://google.github.io/A2A/. Accessed 20 Apr 2026
22. Shi T, Zhu K, Wang Z, Jia Y, Cai W, Liang W, Wang H, Alzahrani H, Lu J, Kawaguchi K, Alomair B, Zhao X, Wang WY, Gong N, Guo W, Song D (2025) PromptArmor: simple yet effective prompt injection defenses. CoRR abs/2507.15219
23. Bhagwatkar R, Kasa K, Puri A, Huang G, Rish I, Taylor GW, Dvijotham KD, Lacoste A (2025) Indirect prompt injections: are firewalls all you need, or stronger benchmarks? arXiv:2510.05244
24. Debenedetti E, Zhang J, Balunović M, Beurer-Kellner L, Fischer M, Tramèr F (2024) AgentDojo source repository: benchmark environments and v1 task definitions, commit 089ed468. https://github.com/ethz-spylab/agentdojo/tree/089ed468. Accessed 26 July 2026

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.